Writer Monad

eason: 7 · Advanced FP · Category: Functional · Level: Advanced · Build with Waq

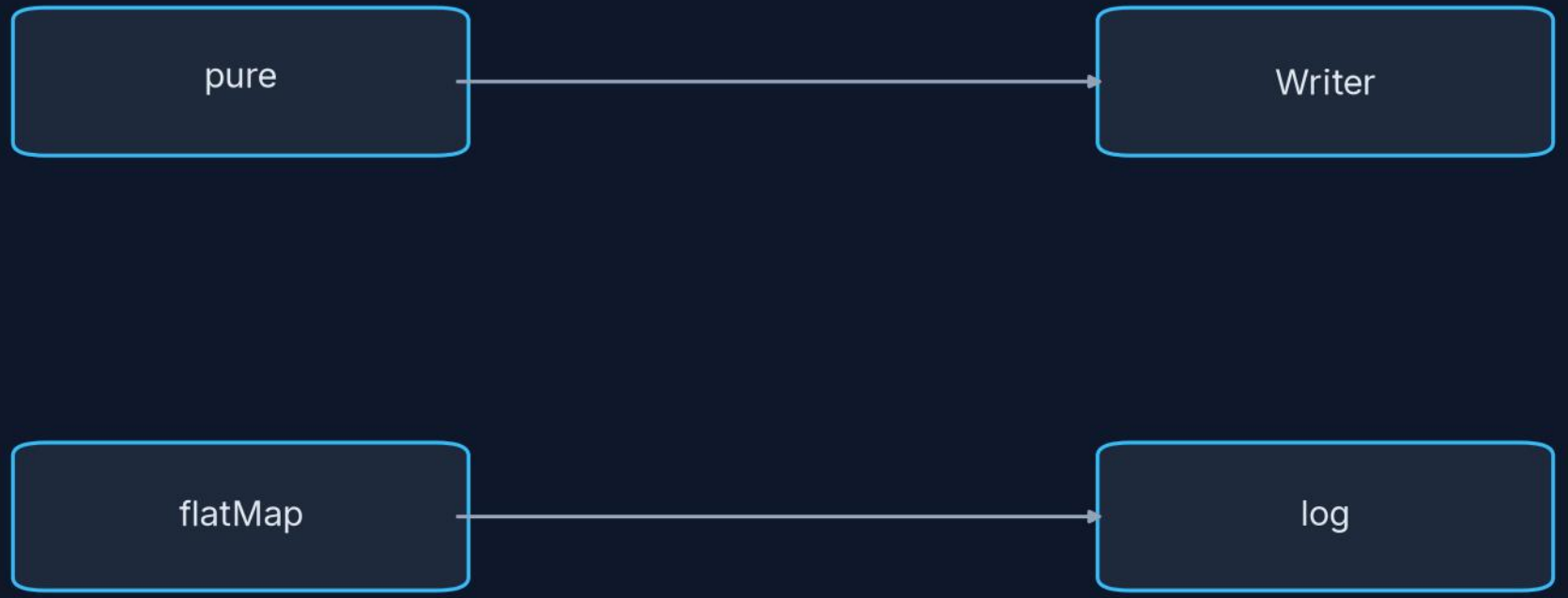
Today's Hook

- Today: Writer Monad as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Writer Monad structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Writer Monad
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/writer/Writer.java
package com.buildwithwaqar.patterns.functional.writer;

import com.buildwithwaqar.patterns.functional.monoid.Monoid;

import java.util.function.Function;

public record Writer<W, A>(W log, A value) {
    public <B> Writer<W, B> map(Function<? super A, ? extends B> f) {
        return new Writer<>(log, f.apply(value));
    }

    public <B> Writer<W, B> flatMap(Monoid<W> monoid, Function<? super A, Writer<W, B>> f) {
        var next = f.apply(value);
        return new Writer<>(monoid.combine(log, next.log), next.value);
    }

    public static <W, A> Writer<W, A> pure(Monoid<W> monoid, A a) {
        return new Writer<>(monoid.empty(), a);
    }

    public static <W> Writer<W, Void> tell(W w) { return new Writer<>(w, null); }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.functional.writer;
import com.buildwithwaqar.patterns.functional.monoid.Monoids;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Writer walkthrough ===");
    var m = Monoids.STRING;
    Writer<String, Integer> w = Writer.<String, Integer>pure(m, 1)
      .flatMap(m, n -> new Writer<>("inc;", n + 1))
      .flatMap(m, n -> new Writer<>("dbl;", n * 2));
    System.out.println("value=" + w.value() + " log=" + w.log());
  }
}
```

Walkthrough Output

```
=== Writer walkthrough ===
value=4 log=inc;dbl;
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Writer Monad
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Writer Monad → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.